

Computer Science Department
Software Engineering & Business Analysis

Bachelor's Thesis

MovieCrush

Movie Tracking Platform with Personalized
Recommendations, Detailed Ratings, and Social Features

Anastasiia Syvak

Supervisor

KSE, Denys Zavhorodnii, dzavhorodnii@kse.org.ua

Expert

KSE, Expert Name, expert@domain.ua

Submission date

16 June 2026

TODOS

1.4 Boxes	11
-----------------	----

- [Personnal todo before marking this thesis as final](#)

Academic Integrity Statement

I, undersigned, hereby declare that this capstone project is the result of my own work.

- All ideas, data, figures and text from other authors have been clearly cited and listed in the bibliography.
- No part of this project has been submitted previously for academic credit in this or any other institution.
- All code, diagrams, and third-party materials are either my original work or are used with permission and properly referenced.
- I have not engaged in plagiarism or any form of academic dishonesty.
- Any assistance received (e.g. from peers, tutors, or online forums) is acknowledged in the acknowledgements section.

I understand that failure to comply with these declarations constitutes academic misconduct and may lead to disciplinary action.

Place, date Kyiv, 03.06.2026

Signature _____

Contents

Acknowledgements	1
Abstract	3
1 Introduction	4
1.1 Basic markup	4
1.2 Images	5
1.3 Tables	5
1.4 Boxes	5
1.5 Citations, Acronyms and Glossary	6
1.6 Code	7
1.7 Context Problem	7
1.8 Objectives	8
1.9 Structure of this report	8
2 Research	9
2.1 Section 1	10
2.2 Section 2	10
2.3 Conclusion	10
3 Design	11
3.1 Section 1	12
3.2 Section 2	12
3.3 Conclusion	12
4 Implementation	13
4.1 Section 1	14
4.2 Section 2	14
4.3 Conclusion	14
5 Validation	15
5.1 Requirements Validation	15
5.1.1 Functional Requirements	15
5.1.2 Non-Functional Requirements	16
5.2 Unit Testing	16
5.2.1 What Is Covered	16
5.2.2 Results	17
5.3 Manual Testing	18
5.3.1 Testing Setup	18
5.3.2 User Feedback	18
5.3.3 Bug Found and Fixed During Testing	19
5.4 ML Model Evaluation	20
5.4.1 Problem Statement	20
5.4.2 Data Flow	20
5.4.3 Synthetic Training Data	20
5.4.4 Iterative Optimization Process	20
5.4.5 Final Results	24
5.4.6 References	25

5.5	Limitations and Future Improvements	26
5.5.1	Current Limitations	26
5.5.2	Planned Improvements	26
6	Conclusion	27
6.1	Project summary	27
6.2	Comparison with the initial objectives	27
6.3	Encountered difficulties	27
6.4	Future perspectives	27
	Glossary	28

Acknowledgements

First of all, I want to thank my parents. For giving me the opportunity to study at such a great university, for their support every single time, for always believing in me when I didn't believe in myself - they always knew I would make it. They have always been my biggest role models in life, and everything I have now is because of them. I also want to thank my mom for being someone I can tell absolutely everything to - every secret, every worry, every thought. And I want to thank my dad for teaching me tennis when I was a child - that's what led me to become the head of the KSE Table Tennis Club and find amazing friends at university, who made these student years the best ones.

I want to thank my grandparents - they were always ready to help me with anything, and were even willing to learn how to code just to make things easier for me. Thank you for the fact that when I had classes until 9 PM, I could get to Akademmistechko in 20 minutes instead of an hour and a half, and thank you for always cooking so much and so well every time I came over, as if it was a wedding and not just my last class of the day. I want to thank my brother for always showing up - early or late - even though he wasn't a student here at that moment, just to help or support me.

I want to thank Veronika Anepska for absolutely everything - for always listening to me, for every party, every conversation, every idea, for believing in each other and supporting each other. Together we built discipline - not just opening TikTok every day to keep our streak alive, but also learning Italian every day, doing yoga every day, setting goals and reaching them together. I'm really happy that this diploma was made with you. I'm sure we have an amazing future ahead, because we're doing everything to make it happen.

I want to thank Bohdana Seheda for sitting next to me in an extra English class in first year and Maryna Chernetska for the fact that we were brought together in first year by the same problem - neither of us understood anything. Those small things helped me find such great friends.

Thank you to Ivan Suprun - after talking to you, everything always feels lighter. Thank you to Marko Hlibovytskyi for always telling me I would be fine when I felt like giving up, and for helping me install Windows when I needed it.

I really want to thank the EBD27 and BE27, and my Verkhovyna company. Thank you to Daria Kozlova for the Integration parties, for Halloween vampires, and for inviting me in my fourth year to get through the hard winter moments and the power outages at Hotel Verkhovyna together. Thank you to Dmytro Shevchuk, Matvii Onyshchenko, Artem Litskevych, Dmytro Vasylchuk, Oleksandr Siryi - you are the best. Every holiday, every party, every memory with you is one of the best in my life. Thank you for every tennis game, every good lunch, every conversation, You are all absolute legends. Thank you to Oleh Skakun for always cheering me up and for driving me home.

Thank you to my friend from Italy, Gabriel, whom I met just before the full-scale invasion started - for helping me feel more confident, for every Eurovision night, for respecting my family and my country, and for the invitation to Italy. Thank you also to my friend from Spain, Markel - I'm sure we will meet someday too.

Thank you to Polina Mamchur, Sofiia Tyshchenko, Sofiia Krotova, Yaroslava Mala and Kateryna Shashkina for every moment we went through together, for finding something positive even during the worst deadline weeks, and for always helping.

I also want to thank Kateryna Pyvovarova for our conversation on the 5th floor of KSE - it was you who came up with the Soulmate feature idea, and I'm really glad that conversation happened.

Thank you to absolutely everyone who came into my life during these years and made it the best it could be - in such difficult times.

Abstract

The abstract serves as a concise summary of your entire thesis, encapsulating key elements on a single page such as:

- *General background information*
- *Objective(s)*
- *Approach and method*
- *Conclusions*

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1 | Introduction

Your introduction serves to introduce the topic of your Bachelor thesis and to arouse the reader's curiosity with an overview. Why it is important and how it is structured, we explain here.

You can consider an introduction as a teaser for your bachelor thesis. You arouse interest and give a foretaste by presenting your motivation, your method and the state of research in your introduction.

Convince your examiners already in the introduction that your Bachelor thesis will be exciting. If your professor starts reading your thesis with anticipation and interest, the chances of getting good grades are higher.

Pay particular attention to the following in your introduction:

- **Introduce the topic** - What characterizes the topic?
- **Introduce the goal** - What do you want to achieve with your thesis?
- **Make the reader curious** - What motivates the reader to read on?
- **Describe the relevance** - Why is this bachelor thesis scientifically relevant?

The introduction should have the following content:

- **Initial situation presentation of the topic** - You introduce the topic with an exciting 'bait'. You provide initial information on the topic and the object of research and explain the current state of research.
- **Relevance of the topic motivation** - You justify the relevance of your topic (scientifically) and place it in the context of your field. In addition, it is often required that you disclose your personal motivation.
- **Objectives** - Your introduction should clearly state what the goal of your paper is and what outcome you hope to achieve upon completion of the bachelor thesis.
- **Method** - You explain the approach and justify the choice of method.
- **Structure of the Bachelor's thesis** - Finally, you give the reader a general overview of your Bachelor's thesis by explaining the structure, showing the red thread and how the research question is answered.



Welcome to the template's introductory chapter! Instead of boring you with lorem ipsum, here's a quick guide to what you can do in Typst and, more specifically, in this template.

Need more? Check out the [Guide to Typst](#).

1.1 Basic markup

Typst lets you create bold, italic, or monospaced text with ease. You can also sprinkle in equations like $e^{i\pi} + 1 = 0$ or even inline code like `fn main() { println!("Hello, World!") }`. And because life is better in color: pink, blue, yellow, orange, green, and more! **Boldly** colorize!

You can also write numbered or unnumbered lists:

- First item
- Second item
 1. First Subitem

- 2. Second Subitem
- Third item

Need equations? Sure! They look great as blocks too:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

1.2 Images

As they say, a picture is worth a thousand words. Let's add one:



Figure 2: Project logo

1.3 Tables

Tables are great for organizing data. From simple to complex, Typst handles them all:

Name	Age	City
Albert Einstein	25	Bern
Marie Curie	22	Paris
Isaac Newton	30	London

Table 1: Simple table

[31:27]			[24:20]		[19:15]	[14:12]	[11:7]		[6:0]	
funct5		aq	rl	rs2	rs1	funct3	rd	opcode		
5				5	5	3	5	7		

Table 2: Complex table

1.4 Boxes

Highlight key points with these fun boxes (and more):



Infobox: For highlighting information.



Ideabox: Share a brilliant idea.



Warningbox: Proceed with caution!



Firebox: This is 🔥 !



Rocketbox: Shoot for the moon!



Todobox: Just do it!

TODO

Personnal todo before marking this thesis as final

1.5 Citations, Acronyms and Glossary

Add citations with @ like [1] or [1, p.7ff] (stored in **/tail/bibliography.bib**).

Acronym terms like **Infotronics (IT)** expand on first use and abbreviate after **IT**. Glossary items such as **Rust Programming Language (Rust)** can also be used to show their description as such: Rust is a modern systems programming language focused on safety, speed, and concurrency. It prevents common programming errors such as null pointer dereferencing and data races at compile time, making it a preferred choice for performance-critical applications.. Acronyms and glossary entries auto-generate at the document's end (defined in **/tail/glossary.typ**).

1.6 Code

Besides writing inline code as such `fn main() { println!("Hello World") }` you can also write code blocks like this:

```

1 fn main() {
2     let ship = Starship::new("USS Rustacean", (0.0, 0.0, 0.0));
3     let destination = (42.0, 13.0, 7.0);
4     let warp = ship.optimal_warp(ship.distance_to(destination));
5
6     println!("👉 {} traveling to {:?} at Warp {:.2}", ship.name, destination,
7 warp);
8     if warp <= 9.0 {
9         println!("🚀 Warp engaged!");
10    } else {
11        println!("⚠ Warp failed!");
12    }
13 }
```

Listing 1: First part of the USS-Rustacean code

or directly from a file

```

1 struct Starship {
2     name: String,
3     position: (f64, f64, f64),
4 }
5
6 impl Starship {
7     fn new(name: &str, position: (f64, f64, f64)) -> Self {
8         Self {
9             name: name.into(),
10            position,
11        }
12    }
13    fn distance_to(&self, dest: (f64, f64, f64)) -> f64 {
14        ((dest.0 - self.position.0).powi(2)
15         + (dest.1 - self.position.1).powi(2)
16         + (dest.2 - self.position.2).powi(2))
17        .sqrt()
18    }
19    fn optimal_warp(&self, distance: f64) -> f64 {
20        (distance / 10.0).sqrt().min(9.0)
21    }
22 }
```

Listing 2: Second part of the USS-Rustacean code from `/resources/code/uss-rustacean.rs`

1.7 Context Problem

Haute École d'Ingénierie (HEI) Rust Rust programs

[1], [1, p.7ff]

```

fn main() {
    println!("Hello World!");
}
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.8 Objectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.9 Structure of this report

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2 | Research

In this section, you perform a State of the Art investigation of your domain to understand what's been built, how it works, and where it falls short. You'll survey existing solutions—tools, frameworks and approaches—then compare them against your project's requirements to justify why a new tool (and its specific feature set) is needed.

A focused Domain Area Research unfolds in these steps:

- 1. Clarify your research questions and functional requirements*
- 2. Collect and review candidate solutions*
- 3. Evaluate each alternative's strengths, weaknesses, maturity and architectural style*
- 4. Group similar approaches into meaningful categories (for example, monolithic vs. microservice, commercial vs. open-source)*
- 5. Pinpoint gaps or missing features that motivate your own design*

This process carries your initial concept through systematic research all the way to a clear, actionable set of requirements for your proposed tool.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Contents

2.1 Section 1	10
2.2 Section 2	10
2.3 Conclusion	10

2.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

3 | Design

In this section you turn your requirements into a concrete engineering blueprint. You'll justify every major architectural choice, visualize structure with C4 diagrams for the first three layers, and map out your runtime topology so that peers can understand—and you can defend—every aspect of your system.

1. Clarify how functional and non-functional requirements drive your high-level architecture
2. List each architectural decision (for example, “We chose microservices to enable independent scaling and deployment”) and explain why it best meets your goals
3. Include a C4 Context diagram showing your system in its environment (users, external systems, data sources)
4. Include a C4 Container diagram breaking the system into deployable units (APIs, web front end, background workers, databases) and annotate communication styles and protocols
5. Include a C4 Component diagram for your core container(s), illustrating key modules, services or libraries and their interactions
6. Describe your deployment topology: physical or cloud hosts, network zones, load-balancing, failover and backup strategies
7. Summarize your technology stack, mapping each tool or framework back to a specific container or component and noting any trade-offs (performance, community support, learning curve)
8. Outline how data flows through the system—including storage models, messaging patterns or API contracts—and note any schema or interface versioning plans
9. Address cross-cutting concerns (Security, Logging, Monitoring, Scalability) and show where they sit in your topology

By walking through Requirements → Decisions → Diagrams → Topology, your Design section becomes a rigorous, evidence-backed foundation for the implementation that follows.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Quod idem licet transferre in voluptatem, ut.

Contents

3.1 Section 1	12
3.2 Section 2	12
3.3 Conclusion	12

3.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

3.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

3.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4 | Implementation

In this section you translate your component-level designs into working code and systems. Focus on the C4 Component layer and on the details needed to show how your design was realized. Include only the most important code snippets that illustrate key patterns or algorithms, rather than full listings.

1. *Describe the development methodology (for example, Agile or test-driven development) used to guide your implementation*
2. *Explain any prototyping or iterative strategies you applied to refine components before full-scale coding*
3. *Summarize coding standards, naming conventions and architectural patterns followed in your codebase*
4. *Present critical code snippets or configuration templates that highlight how core components were implemented (for example, key classes, interfaces or algorithms)*
5. *Detail your testing approach and quality assurance measures (unit tests, integration tests, coverage metrics)*
6. *Note any performance optimizations or profiling results for components that were bottlenecks*
7. *Outline your deployment and configuration management process for component artifacts (containerization, CI/CD pipelines)*
8. *Highlight documentation deliverables (API references, inline comments, architecture decision records) that support future maintenance*

This section demonstrates how each component specification becomes actual, maintainable code—closing the loop from design to implementation.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Quod idem licet transferre in voluptatem, ut.

Contents

4.1 Section 1	14
4.2 Section 2	14
4.3 Conclusion	14

4.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

5 | Validation

5.1 Requirements Validation

The tables below list all key requirements, how each one was tested, and the result.

5.1.1 Functional Requirements

Requirement	How tested	Result
Personalized recommendations based on user behavior	Offline evaluation - ROC AUC 0.8771	Pass
Cold start for new users	Manual testing with a new account	Pass
Onboarding flow for cold start signal collection	Manual testing with a new account	Pass
Ratings - overall (1-10) and detailed (5 criteria)	Manual testing + DB check	Pass
Search for movies, series, actors, and users	Manual live search testing	Pass
Lists - Watched, Watchlist, Favorites, custom	Manual testing + DB check	Pass
Social features - follows, friends' ratings	Manual testing between two accounts	Pass
Soulmate - finding a user with similar taste	Unit tests (soulmate_math) + manual testing	Pass
Wrapped - personal yearly statistics	Unit tests (wrapped) + manual testing	Pass
Comments with anonymous and spoiler options	Manual testing of all comment modes	Pass
Episode tracking for series	Unit tests (episode_helpers) + manual testing	Pass
Mood tagging after watching	Manual testing + DB check	Pass
Actor page with filmography	Manual testing	Pass
Movie page with full content	Manual testing	Pass
Registration and login with validation	Unit tests (auth_validators) + manual testing	Pass

Requirement	How tested	Result
Email confirmation and password recovery	Manual flow testing	Pass
JWT authentication with refresh token rotation	Unit tests + middleware type check	Pass
Challenges and premiere notifications	-	Planned
Public iOS distribution	-	Requires Apple Developer account

Table 3: Functional requirements - testing method and result

5.1.2 Non-Functional Requirements

Requirement	How tested	Result
Rate limiting - 200 requests per minute per IP	Response headers check	Pass
HTTPS for all public endpoints	Verified via Render dashboard	Pass
Two-level cache - LRU in-memory + PostgreSQL	Code review + manual response time check	Pass
CI/CD pipeline with automated checks	GitHub Actions run on every push	Pass
Password hashing with bcrypt	Code review - no plain text passwords stored	Pass
Health check endpoint	GET /health returns status ok	Pass

Table 4: Non-functional requirements - testing method and result

5.2 Unit Testing

The backend uses Jest together with ts-jest for TypeScript support. All tests live in the `__tests__` folder and run automatically in the CI pipeline on every push. The focus is on critical business logic - the parts of the system where a bug directly affects data correctness or security.

5.2.1 What Is Covered

5.2.1.1 `soulmate_math.test.ts`

Covers all the math behind the Soulmate algorithm - cosine similarity, Jaccard similarity, `cosineSimilarityFromCounts`, `weightedSum`, `buildRatingVectors`, and `isRecomputeThrottled`.

Special attention is paid to edge cases: empty vectors, zero norms (to avoid division by zero), vectors of different lengths, and cooldown logic with different time intervals. This matters because any bug in these functions directly affects the quality of user matching.

Examples of what is checked:

- two identical vectors produce similarity = 1.0
- two orthogonal vectors produce similarity = 0
- weightedSum with all zero metrics returns 0
- cooldown returns true if less than 24 hours have passed, false if more or if lastComputedAt is null

5.2.1.2 als_service.test.ts

Covers helper functions in the recommendation service: getCfRatio, filterValidItems, sliceToLimit, cacheRowToAlsItem, mediaTypeFromTmdb, and releaseYearFromTmdb.

A key check is the threshold that switches between cold start and the ALS pipeline via getCfRatio - when watchedCount is 50 or more, the higher CF ratio (0.7) is returned; below that, the lower one (0.5). filterValidItems is also tested to confirm it correctly skips items without a title and preserves the order coming from ALS.

5.2.1.3 wrapped_test.ts

Covers the calculation of personal yearly statistics: top genre, top actor based on vote count, cinema ritual (the most common watch time of day), and total watch time in minutes.

5.2.1.4 auth_validators.test.ts

34 tests covering input validation for registration and login. Checks include email format (with and without subdomains, dots, special characters), username rules (minimum and maximum length, allowed characters), and password requirements (minimum length, presence of digits and letters).

This is important from a security perspective - the validators are the first line of defense before any data reaches the backend.

5.2.1.5 episode_helpers.test.ts

27 tests for the parseTmdbId helper, which solves a real bug found during development: PostgreSQL stores BIGINT but the pg driver returns it as a string to avoid losing precision on 64-bit values. The helper accepts both numeric and string formats and returns the correct number or null for invalid input.

Tested cases include: regular numbers, numeric strings, negative numbers, strings with letters, null, undefined, and empty string.

5.2.1.6 tmdb_helpers.test.ts

Helper functions for working with TMDB API responses - detecting media type (movie or tv) and parsing release dates.

5.2.2 Results

All tests pass. The CI pipeline runs them automatically via **npm test** on every push to main and feature branches - if any test fails, the build does not pass and deployment does not happen.

One limitation worth noting: there are no integration tests for complex end-to-end flows such as the full recommendation pipeline, since those require live database and CF service connections. Is noted as an area for future improvement.

5.3 Manual Testing

5.3.1 Testing Setup

The app was tested manually throughout the development process. Three external testers used the app through Expo Go on my personal Android device. An Android APK was also built via EAS Build for direct distribution without the Play Store, though the main testing session was conducted through Expo Go. iOS is compatible but requires an Apple Developer account for public distribution, so it was demonstrated on a personal device only.

5.3.2 User Feedback

All three testers praised the interface and said the app felt comfortable and polished to use. All feedback was given in Ukrainian and is translated here for this document.

Tester 1:



I really like that this app has a dark theme, a light one would definitely not work here. As soon as you open the app you start thinking about what you want to watch, you feel comfortable, and you immediately imagine the atmosphere of a cinema or a cozy evening at home under a warm blanket with a series and a cup of tea.

They also noted that the social layer was a highlight - especially seeing friends' ratings directly on a movie or episode page, and that Wrapped analytics felt exciting and natural for a generation that loves tracking everything.

Two improvement requests came up: the “More Like This” section felt like it could show better recommendations, and they would like to see streaming availability directly on the movie page (for example, whether a film is on Netflix or Kyivstar TV). The “More Like This” issue was already addressed by switching from the TMDb `/similar` endpoint to `/recommendations` with a fallback. Streaming availability is planned for future development.

Tester 2:



The interface is very intuitive - you really understand where to find everything, as if the app was built from a template, the developer clearly understood what goes where. The analytics part amazed me. On Spotify I wait to see my Wrapped like it's a holiday, I love sharing it in my Instagram stories so everyone knows how cool I am. I would really love to share film analytics in stories too. The recommendations are great - I saw films in my recommendations that I had already watched and loved, I just hadn't added them to my watched list in the app yet. And I watched one film from the list and gave it 9 out of 10. So I am very happy.

This was a strong real-world signal that the hybrid ALS + Gemini pipeline was working as intended - without any explanation of how the system works, the tester independently noticed that the recommendations matched their actual taste.

Tester 3:



I really like the colors - they don't hurt your eyes and they match each other. I was pleasantly surprised by the onboarding because I use Letterboxd and I have never seen anything like it there. I am really glad that I can immediately enter what I like and get recommendations based on that. I love that I can go to an actor's page and see where they appeared - sometimes I get really into an actor and now instead of googling I just go to their page from a film I watched and browse their filmography. And the fact that there is a split between Acting and Crew tabs means I can immediately open just the Acting tab. I also like that the rating is shown right on the poster so I don't have to open the film to see it. And I really appreciated the filter in recommendations especially the tab "russia is a terrorist state" - that is a nice reminder. In the future I would love a permanent filter where I can set that I never want to see content from a specific country in my recommendations at all.

5.3.3 Bug Found and Fixed During Testing

During the session with Tester 3, a performance issue was discovered. Opening a profile list with 30 films triggered 30 parallel requests to TMDB through the backend proxy. Switching between lists quickly exceeded the rate limit and returned 429 Too Many Requests errors.

The fix was a new **POST /tmdb/media/batch** endpoint - a single request that reads all metadata from **tmdb_media_cache** in one SQL query using **WHERE tmdb_id = ANY(...)** and only fetches missing entries from TMDB. Instead of 30 requests, the list now loads

with 1. The cache is shared across users, so popular films are fetched from TMDB only once for the entire service.

5.4 ML Model Evaluation

5.4.1 Problem Statement

The goal was to build a collaborative filtering recommendation system that, based on user behavior (watched films, ratings, favorites), produces personalized recommendations - films the user is likely to want to watch next.

The initial plan was to use LightFM - a more flexible library with support for content-based and collaborative hybrid filtering. However, LightFM has no official Windows support, which made development impossible in the available environment. The **implicit** library was chosen as a replacement because it implements the same ALS algorithm from the original Hu, Koren, Volinsky (2008) paper, has full Windows support, and supports BM25 weighting and confidence-aware training.

5.4.2 Data Flow

The full pipeline from raw data to recommendations:

- PostgreSQL (user_movie_actions, user_detailed_ratings)
- build_matrix() - sparse CSR matrix (users on items)
- BM25 weighting (reduces influence of popular items and superfan users)
- ALS training - user and item embeddings
- .recommend(user_id) - top-N items
- HTTP response - Node.js backend - mobile app

5.4.3 Synthetic Training Data

Since there were not enough real users at the time of evaluation, the training dataset was generated synthetically.

The initial generation script (**generate_ratings_for_all_users.py**) split 208 users into 8 fixed genre profiles and selected films randomly from large pools. The problem was that two users with the same profile would watch completely different random films - the overlap between them was tiny, and the model could not learn the pattern “users who watched X also watch Y” because such pairs did not exist in the data.

A new script **generate_realistic_data.py** was built with a fundamentally different approach: films were grouped into 17 semantic clusters (marvel_mcu, john_wick_franchise, nolan_films, tarantino, ghibli, pixar, lotr_hobbit and others) where films inside a cluster are genuinely related - same franchise, director, or style. Each user has 2-4 primary clusters with high ratings, 1-3 affiliated clusters through a cross-cluster affinity graph, 1-2 disliked clusters with low ratings, and 2-4 random films as noise. Real user accounts used for manual testing were protected and excluded from synthetic data generation.

5.4.4 Iterative Optimization Process

Development went through 5 iterations, each producing a measurable improvement in metrics. The approach was: baseline, problem, hypothesis, experiment, measurement.

5.4.4.1 Iteration 1 – Baseline

Configuration: 208 synthetic users, 315 films, 10K interactions. ALS with factors=32, iterations=20, regularization=0.1. No BM25 weighting. Ratings used directly as confidence weights.

```
Користувачів протестовано: 208
Hit Rate@1: 0.0000 (0.0%)
Hit Rate@5: 0.0096 (1.0%)
Hit Rate@10: 0.0673 (6.7%)
Precision@10: 0.0673 (6.7%)
```

Figure 3: Iteration 1 - baseline results

Four problems were found: incorrect LOO methodology where a separate model was trained per user (208 models), missing BM25 weighting, wrong confidence interpretation where ratings were used directly as weights instead of being scaled through the confidence parameter alpha as the original ALS paper requires, and dislikes added as weak positive signals with weight 0.1 which the model interpreted as mild interest rather than rejection.

5.4.4.2 Iteration 2 - Algorithmic Fixes

Changes: single model trained on all users (correct LOO), BM25 weighting added, alpha parameter introduced, dislikes fully removed from the training matrix and used only as a post-filter at inference time, structured preference scoring introduced.

```
(venv) PS C:\Users\Anastasiia\Documents\moviecrush\lightfm_service> python evaluate.py
Building matrix...
Fetching interactions...
Matrix shape: (208, 310)
Sparsity: 84.57%
Total interactions: 9935
Shape: (208, 310), NNZ: 9935
100%|████████████████████████████████████████████████████████████████████████████████| 30/30 [00:00<00:00, 499.46it/s]

Users tested: 208
Hit Rate@1: 0.48%
Hit Rate@5: 7.69%
Hit Rate@10: 17.31%
Hit Rate@20: 32.69%
MRR (found): 0.8843
Avg rank when found: 24.8
```

Figure 4: Iteration 2 - results after algorithmic fixes. Hit Rate@10 tripled.

5.4.4.3 Iteration 3 - Realistic Data Generation

After switching to the cluster-based generation script:

```
(venv) PS C:\Users\Anastasiia\Documents\moviecrush\lightfm_service> python evaluate.py
Building matrix...
Fetching interactions...
Matrix shape: (208, 265)
Sparsity: 89.78%
Total interactions: 5633
Shape: (208, 265), NNZ: 5633
100%|████████████████████████████████████████████████████████████████████████████████| 30/30 [00:00<00:00, 499.46it/s]

Users tested: 208
Hit Rate@1: 4.33%
Hit Rate@5: 28.85%
Hit Rate@10: 46.15%
Hit Rate@20: 65.38%
MRR (found): 0.1956
Avg rank when found: 13.7
```

Figure 5: Iteration 3 - results after realistic data generation

Key finding: data quality turned out to be a more critical factor than all algorithmic fixes combined. Hit Rate@10 jumped from 17% to 46% - a larger gain than all previous improvements together.

5.4.4.4 Iteration 4 - Full Evaluation (Classification + ROC AUC)

A full set of metrics was added: Precision@K, Recall@K, F1@K, NDCG@K, and ROC AUC.

ROC AUC methodology for collaborative filtering. There is no standard classification scenario in CF, so a negative sampling approach was used: for each test user, 1 known relevant item is taken and 50 random items the user has not watched are sampled as negatives. The model scores all 51 items and the ROC curve is built on (y_{true} , y_{scores}).

K	Hit Rate@K	NDCG@K
1	4.33%	0.0433
5	28.85%	0.1668
10	46.15%	0.2215
20	65.38%	0.2704
MRR: 0.1956		
Average rank (when found): 13.74		

Figure 6: Iteration 4 - Hit Rate and NDCG results

K	Precision@K	Recall@K	F1@K
1	0.1923	0.0082	0.0157
5	0.4346	0.0922	0.1505
10	0.5721	0.2415	0.3342
20	0.6704	0.5544	0.5954

Figure 7: Iteration 4 - Precision, Recall, F1 and ROC AUC = 0.8946

5.4.4.5 Iteration 5 - Hyperparameter Tuning (Grid Search)

A systematic grid search over 60 combinations was run to find the optimal configuration instead of manual tuning.

Search space: factors {12, 16, 20, 24, 32}, alpha {20, 40, 60, 80}, regularization {0.1, 0.5, 1.0}.

Each combination was trained with fixed random_state=42 on the same train/test split and evaluated on the same test pairs.

```

Building matrix...
Fetching interactions...
Matrix shape: (208, 265)
Sparsity: 89.78%
Total interactions: 5633
Shape: (208, 265), NNZ: 5633

Users to test: 208
Total combinations to test: 60

```

factors	alpha	reg	HR@5	HR@10	HR@20	MRR	AvgRank	
12	20	0.1	23.56%	40.38%	64.42%	0.2087	14.4	[1/60]
12	20	0.5	23.08%	43.27%	66.83%	0.1882	14.3	[2/60]
12	20	1.0	25.00%	40.87%	64.42%	0.1899	15.1	[3/60]
12	40	0.1	17.79%	37.98%	62.02%	0.1554	15.8	[4/60]
12	40	0.5	18.27%	31.73%	54.81%	0.1613	18.2	[5/60]
12	40	1.0	17.79%	39.42%	57.69%	0.1345	18.4	[6/60]
12	60	0.1	21.63%	35.10%	59.62%	0.1873	17.2	[7/60]
12	60	0.5	21.15%	31.73%	50.00%	0.1734	18.7	[8/60]
12	60	1.0	20.67%	32.69%	50.48%	0.1728	17.9	[9/60]
12	80	0.1	21.15%	35.58%	56.73%	0.1794	16.8	[10/60]
12	80	0.5	15.87%	29.33%	47.60%	0.1578	18.9	[11/60]
12	80	1.0	22.60%	30.29%	47.12%	0.1921	18.1	[12/60]
16	20	0.1	30.29%	47.12%	68.75%	0.1986	13.5	[13/60]
16	20	0.5	28.85%	42.79%	70.67%	0.1859	14.1	[14/60]
16	20	1.0	27.40%	50.96%	70.19%	0.2122	13.1	[15/60]
16	40	0.1	23.08%	37.50%	60.58%	0.1578	17.0	[16/60]
16	40	0.5	28.85%	46.15%	65.38%	0.1956	13.7	[17/60]
16	40	1.0	26.44%	43.27%	59.62%	0.1985	15.1	[18/60]
16	60	0.1	22.60%	38.94%	56.25%	0.1861	17.4	[19/60]
16	60	0.5	22.60%	37.50%	57.21%	0.1811	16.7	[20/60]
16	60	1.0	26.92%	38.94%	61.06%	0.2039	15.0	[21/60]
16	80	0.1	18.75%	36.54%	55.29%	0.1663	18.0	[22/60]
16	80	0.5	21.63%	33.17%	49.52%	0.1987	17.4	[23/60]
16	80	1.0	25.48%	37.98%	54.81%	0.2012	16.6	[24/60]
20	20	0.1	29.33%	51.92%	71.63%	0.2044	11.3	[25/60]
20	20	0.5	30.29%	50.96%	72.60%	0.2104	12.1	[26/60]

Figure 8: Grid search results - 60 hyperparameter combinations

Key observations: all top-5 configurations by HR@10 and MRR use alpha=20 - lower than the initial value of 40. For a sparse matrix, lower confidence weighting works better because it does not overfit on popular items. factors=24 showed the best balance. regularization=1.0 was optimal - on a small matrix, stronger smoothing is needed.

Selected configuration: **factors=24, alpha=20, regularization=1.0.**

5.4.5 Final Results

K	Hit Rate@K	NDCG@K
1	10.58%	0.1058
5	32.69%	0.2174
10	51.92%	0.2802
20	63.94%	0.3115

MRR: 0.2639
Average rank (when found): 12.11

Figure 9: Final results - Hit Rate and NDCG after all iterations

K	Precision@K	Recall@K	F1@K
1	0.2885	0.0128	0.0244
5	0.5837	0.1254	0.2042
10	0.7413	0.3132	0.4336
20	0.8209	0.6749	0.7268

Figure 10: Final results - Precision, Recall, and F1

ROC AUC: 0.8771
(1.0 = perfect, 0.5 = random, your value)

ROC curve points (sample):

Threshold	FPR	TPR
inf	0.0000	0.0000
0.9315	0.0187	0.2933
0.7465	0.0486	0.5625
0.3932	0.1292	0.8029
-1.3861	1.0000	1.0000

Figure 11: Final ROC AUC = 0.8771

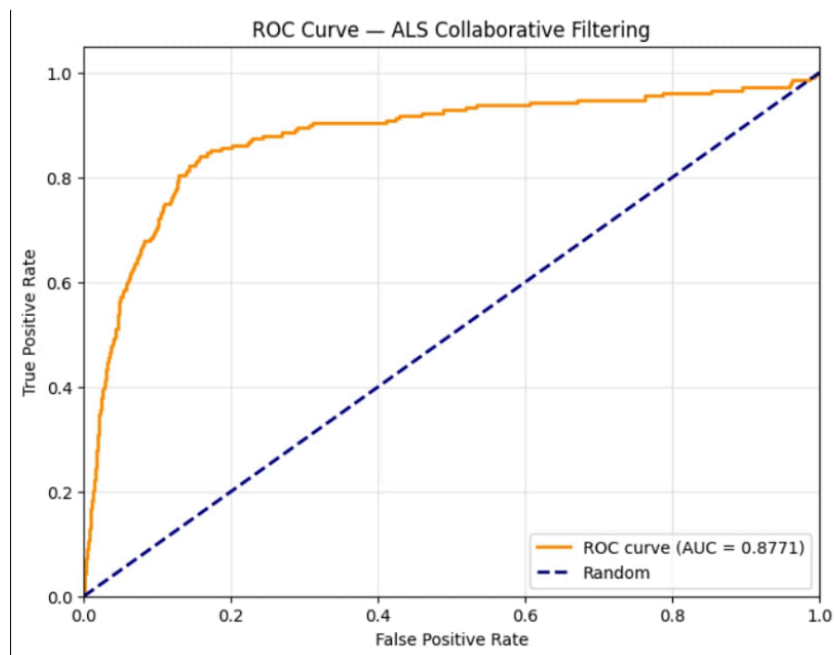


Figure 12: ROC Curve - ALS Collaborative Filtering. AUC = 0.8771 is substantially above the random baseline of 0.5

ROC AUC = 0.8771 means the model correctly ranks a relevant film above a random irrelevant one in 87.71% of cases.

K	Hit Rate@K	Precision@K	NDCG@K
1	10.58%	28.85%	0.1058
5	32.69%	58.37%	0.2174
10	51.92%	74.13%	0.2802
20	63.94%	82.09%	0.3115

Table 5: Final ALS model metrics after all iterations. MRR: 0.2639. Average rank when found: 12.11

5.4.6 References

The following academic sources informed the methodology and implementation of the collaborative filtering module:

1. Hu, Y., Koren, Y., & Volinsky, C. (2008). **Collaborative Filtering for Implicit Feedback Datasets**. Proceedings of ICDM 2008. [ACM Digital Library](#)
2. Frederickson, B. (2022). **implicit: Fast Python Collaborative Filtering for Implicit Feedback Datasets**. [GitHub](#)
3. Robertson, S., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. Foundations and Trends in Information Retrieval. [ResearchGate](#)

5.5 Limitations and Future Improvements

5.5.1 Current Limitations

Synthetic data for ML evaluation. The ALS model was evaluated on a generated dataset that models realistic viewing patterns, but not on real users. Production metrics may differ - either better (more data means better embeddings) or worse (real taste patterns are more complex than synthetic ones).

Small interaction matrix. The current training matrix is 2080x265 - very small for a production system. ALS works better on larger matrices where there are more cross-user signals. As the user base grows, the metrics should improve.

Manual model retraining. The model is currently retrained manually. In production, automated cron jobs are needed to keep the model up to date with new user data.

No TTL on tmdb_media_cache. Movie metadata is stored in the cache without an expiry time. If TMDB updates a poster or any other information, the cache will not pick up the change. A periodic refresh job with a weekly TTL is planned as a future improvement.

iOS distribution. Public distribution on iOS requires an Apple Developer account. At the moment the iOS version is only shown on a personal device through Expo.

Email verification in the test environment. The free Resend plan without a custom domain can only send emails to the account owner's address. For testing purposes, email verification was made optional - the mechanism is kept in the codebase and ready for production use.

Limited external testing. Three testers had access to the app through Expo Go on my device. Testing across a wider range of devices and OS versions remains a task for the next stage.

5.5.2 Planned Improvements

- Automated ALS model retraining on a schedule (cron job)
- Challenges and gamification - achievements, badges, and viewing goals
- Push notifications for premiere dates of tracked series
- Language switch in the app (Ukrainian/English)
- Instagram Stories share templates for Wrapped and Soulmate results
- Public iOS release after getting an Apple Developer account
- Custom domain for Resend with full email verification for all users
- Periodic tmdb_media_cache refresh with a weekly TTL
- Affiliate partnerships with streaming platforms (JustWatch model)
- Monetization through MovieCrush Pro subscription

6 | Conclusion

Conclusion

In this final section you bring together your work and reflect on its impact. Keep it concise, restating key points without introducing new information:

- 1. Project Summary: Briefly recap objectives, methodology and principal results*
- 2. Alignment with Objectives: Discuss how outcomes meet initial goals, referencing requirements and design aims*
- 3. Lessons Learned and Challenges: Note any obstacles and how they informed improvements*
- 4. Limitations: Acknowledge features or scenarios beyond this scope and clearly state current system boundaries*
- 5. Future Work: Suggest practical enhancements or research directions building on your findings*

Avoid introducing new concepts here; refer readers to the Discussion for deeper analysis.

6.1 Project summary

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium sit nunc nisl. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

6.2 Comparison with the initial objectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium sit nunc nisl. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

6.3 Encountered difficulties

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium sit nunc nisl. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

6.4 Future perspectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium sit nunc nisl. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Glossary

Programming Language

Rust – Rust Programming Language: Rust is a modern systems programming language focused on safety, speed, and concurrency. It prevents common programming errors such as null pointer dereferencing and data races at compile time, making it a preferred choice for performance-critical applications. [6](#), [7](#)

University

HEI – Haute École d'Ingénierie [7](#)

IT – Infotronics [6](#)

Bibliography

- [1] S. Zahno *et al.*, “Dynamic Project Planning with Digital Twin,” *Frontiers in Manufacturing Technology*, vol. 3, May 2023, doi: [10.3389/fmtec.2023.1009633](https://doi.org/10.3389/fmtec.2023.1009633).

A | Appendix

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.